Scaling Zero-Knowledge Proofs on Ergo: Native secq256k1 Simulation and eUTXO Pipelining for Curve Trees

A. Shannon

February 2026

Abstract

Zero-knowledge accumulators requiring recursive cryptographic proofs construct a significant bottleneck for base-layer blockchain execution environments. Specifically, the Curve Tree construction—which achieves practical and transparent zero-knowledge accumulators—mandates the evaluation of hash functions over a 2-chain of elliptic curves. This paper demonstrates a novel methodology for evaluating highly complex zero-knowledge accumulator membership proofs natively on the Ergo Layer 1 blockchain. By leveraging Ergo 6.0's newly introduced `UnsignedBigInt` modular arithmetic operators, we prove the feasibility of simulating elliptic curve operations on `secq256k1` ($\mathcal{E}_2$), thereby completing the 2-chain required to recursively pair with Ergo's native `secp256k1` ($\mathcal{E}_1$). Furthermore, to overcome the hard $1,000,000$ JitCost block limitation when verifying the necessary Generalized Bulletproofs (which we estimate at $\approx 1.2 \times 10^9$ JitCost for 2^{20} leaves under linear verification complexity), we propose *Execution Pipelining*. This deterministic, eUTXO-native design pattern serializes the Multi-Scalar Multiplications (MSM) across $\approx 1,200$ sequential blocks, effectively utilizing the consensus network as a decentralized Turing tape without requiring optimistic wrappers, challenge periods, or external watchers.

---

1. Introduction

The integration of highly scalable privacy and verifiable computation on public blockchains is fundamentally constrained by the high computational overhead of zero-knowledge proof systems. While account-based models like Ethereum have addressed this by utilizing specialized precompiles (e.g., EIP-196), such additions ossify the protocol layer.

The Ergo blockchain, predicated on the extended Unspent Transaction Output (eUTXO) model and the Sigma constraint language (ErgoScript), operates under a strict computational cost regime mapped to `JitCost`. The absence of natively supported pairing-friendly curves (e.g., alt-BN128) historically restricted Ergo's capacity to process advanced zero-knowledge accumulators.

In this work, we present a synthesis of cryptographic simulation and topological state continuations allowing Ergo to support massive-scale accumulators—specifically Curve Trees [1]—requiring zero protocol modifications.

2. Cryptographic Preliminaries

2.1 The 2-Chain Requirement

A Curve Tree is a zero-knowledge membership proof system built over a tree of algebraic commitments. To efficiently verify a Curve Tree without highly expensive non-native arithmetic (like bit decomposition), the prover computes Pedersen hashes on alternating elliptic curves [1]. The base field output of a hash on $\mathcal{E}_1$ must perfectly match the scalar field input for the subsequent hash on $\mathcal{E}_2$.

Formally, a full **2-cycle** requires two prime-order elliptic curves $\mathcal{E}_1(\mathbb{F}_{p_1})$ and $\mathcal{E}_2(\mathbb{F}_{p_2})$ with respective group orders n_1 and n_2, such that:

$$p_1 = n_2 \quad \text{and} \quad p_2 = n_1$$

Curve Trees, however, require only a **2-chain** — a weaker condition where the field-order relationship holds in one direction per tree level, i.e., $p_2 = n_1$ suffices [1]. The concept of mutually paired cycles of elliptic curves was originally formalized by Ben-Sasson et al. (2014) [7] for recursive proof composition, and was later popularized in deployment by Halo [4]. Ergo natively supports exactly one curve: $\mathcal{E}_1 =$ `secp256k1`. The necessary sister curve to complete the 2-chain is $\mathcal{E}_2 =$ `secq256k1`, which evaluates the identical Weierstrass equation $y^2 = x^3 + 7$ over the prime field $\mathbb{F}_{p_2}$ where $p_2 = n_{secp}$.

2.2 Generalized Bulletproofs

Curve Trees utilize Generalized Bulletproofs—an extension of Inner Product Arguments [5]—to prove the satisfying arithmetic circuit of the hash tree path constraints. Bulletproofs [2] require no trusted setup and rely purely on the discrete logarithm assumption, rendering them ideal for Ergo's transparent ethos.

3. Methodology: Simulating $\mathcal{E}_2$ over $\mathbb{F}_{p_2}$

With the deployment of Ergo Node v6.0 (EIP-0050) [3], ErgoScript gained the `UnsignedBigInt` operational type, fundamentally decoupling arithmetic from the constraints of 64-bit integer overflows. This introduced modular arithmetic opcodes (`plusMod`, `subtractMod`, `multiplyMod`, and `modInverse`) that evaluate in $O(1)$ time strictly within Ergo's *JitCost accounting model*, despite their underlying arbitrary-precision computational complexity.

Because $p_{secq} = n_{secp}$, we can execute point arithmetic for the $\mathcal{E}_2$ curve entirely within ErgoScript's `UnsignedBigInt` context by parameterizing the modulus to `CryptoConstants.groupOrder`.[1]

3.1 Point Addition Formulation

Given two distinct affine points $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ on $\mathcal{E}_2$, the point addition $R = P + Q$ is computed natively on L1 via:

$$\lambda \equiv (y_2 - y_1) \cdot (x_2 - x_1)^{-1} \pmod{p_{secq}}$$

$$x_3 \equiv \lambda^2 - x_1 - x_2 \pmod{p_{secq}}$$

$$y_3 \equiv \lambda \cdot (x_1 - x_3) - y_1 \pmod{p_{secq}}$$

In ErgoScript, this requires exactly 1 `modInverse`, 3 `multiplyMod`, and 6 `subtractMod` operations.

3.2 Point Doubling Formulation

For $P = Q$, the point geometry necessitates the tangent line derivative $R = 2P$:

$$\lambda \equiv 3x_1^2 \cdot (2y_1)^{-1} \pmod{p_{secq}}$$

$$x_3 \equiv \lambda^2 - 2x_1 \pmod{p_{secq}}$$

$$y_3 \equiv \lambda \cdot (x_1 - x_3) - y_1 \pmod{p_{secq}}$$

3.3 The eUTXO Affine Anomaly

In standard elliptic curve cryptography (and EVM precompiles), evaluating internal scalar multiplication loops using Affine coordinates (x, y) is considered highly inefficient because calculating the modular inverse (x^{-1}) consumes $\approx 80\times$ more CPU cycles than a standard multiplication. Standard practice utilizes Jacobian or Projective coordinates (X, Y, Z) to delay the inversion until the final step.

However, Ergo's JIT compiler prices `UnsignedBigInt` operations in a way that creates a stark architectural anomaly:[2] * `modInverse` = 150 JitCost * `multiplyMod` = 40 JitCost * `plusMod` / `subtractMod` = 30 JitCost

Evaluating the Affine point addition formulated above requires $150 + 3(40) + 6(30) =$ **450 JitCost**. Conversely, executing a standard Jacobian point addition requires approximately 12 multiplications and 7 additions/subtractions, yielding $\approx$ **690 JitCost**.

Ergo's unique opcode pricing flips standard cryptographic dogma on its head, rendering Affine geometry strictly cheaper for on-chain evaluation. This empirical reality uniquely justifies operating on the $\mathcal{E}_2$ simulated curve entirely in Affine space.

3.4 Complexity and the Pippenger Paradox

Based on this empirical baseline, evaluating a complete 256-bit scalar multiplication via standard double-and-add consumes an estimated **148,000 JitCost**.

Verifying a Curve Tree of depth $d = 20$ (capable of supporting $2^{20} \approx 1,000,000$ leaves) produces an arithmetic circuit of $\approx 8,192$ constraints. The Bulletproof verifier performs $\lceil \log_2(n) \rceil$ recursive IPA rounds to fold the proof, but the dominant verification cost arises from the final $\mathcal{O}(n)$ Multi-Scalar Multiplication used to eval-

uate the polynomial identity against the full set of generators [1, 2]. Evaluating this MSM sequentially over the simulated $\mathcal{E}_2$ curve equals roughly **1.2 Billion JitCost**.

While optimized algorithms like Pippenger's bucket method yield $O(N/\log N)$ complexity, ErgoScript's purely functional `Coll` collections lack the mutable arrays required to execute dynamic bucket-sorting efficiently. Specifically, functional simulation of Pippenger bucket sorting introduces $O(N \cdot \log N)$ `Coll` reallocation overhead — each bucket operation requires copying the entire collection — estimated at $3\text{-}5\times$ the theoretical speedup, netting no practical gain on-chain. Therefore, the pipeline must assume linear verification complexity, shattering Ergo's theoretical block limit of $1,000,000$ JitCost by three orders of magnitude.

4. Execution Pipelining: The eUTXO Turing Tape

To resolve this computational ceiling without deferring to Optimistic Rollup architectures (which introduce Game Theoretic assumptions, collateral bonds, and arbitrary Challenge Periods), we propose **Execution Pipelining**.

Execution Pipelining exploits the determinism of the eUTXO model [6] to force the Layer 1 consensus nodes to evaluate every mathematical constraint natively, completely preserving cryptographic trustlessness.

4.1 Structured Reference String via Global AVL Tree

The Bulletproof verification circuit requires access to $n = 8,192$ fixed generator points $\vec{g}$ and $\vec{h}$ constituting the Structured Reference String (SRS). These generators total ≈ 530 KB of data, violating Ergo's absolute transaction size limit (≈ 96 KB) and single-box limit (≈ 4 KB).

Critically, these generators are *fixed public parameters* of the circuit—not user-specific data. They are deployed once at protocol genesis into a global AVL Tree (Authenticated Dictionary) whose 33-byte Digest is embedded in the protocol's bootstrap configuration.

The actual Bulletproof proof transcript submitted by a prover consists only of $\approx \lceil \log_2(n) \rceil$ curve points (L_i, R_i) and a small number of scalars—totaling $\approx 1\text{-}2$ KB, which fits natively inside a single Ergo transaction's registers.

During pipelined execution, each Tx_k utilizes Ergo's `CONTEXT.dataInputs` to provide Merkle inclusion proofs for the specific subset of fixed generators needed for that block's mathematical chunk from the global Genesis AVL Tree. The smart contract dynamically computes the required Fiat-Shamir challenge scalars on-the-fly from the proof transcript stored in the registers. Note that AVL Tree Merkle proof verification itself consumes non-trivial JitCost per lookup; the per-block budget of $\sim 900{,}000$ JitCost must therefore account for both MSM computation *and* proof verification overhead, which may reduce the effective chunk size by an estimated $5\text{-}10\%$.

4.2 Deterministic State Continuation

Because an eUTXO allows a smart contract to preserve its operational state into the subsequent output's registers, we segment the 1.2×10^9 JitCost MSM execution into $N \approx 1,200$ discrete transactions.

Let chunk k of the proof execution have an accumulator state Acc_k. A `ProofInProgress` box is defined strictly by the following data structure:

- `R4` (`GroupElement`): $Acc_Point_{\mathcal{E}_1}$ — valid here because $\mathcal{E}_1$ is Ergo's native `secp256k1`
- `R5` (`UnsignedBigInt`): Acc_Scalar
- `R6` (`Tuple[UnsignedBigInt, UnsignedBigInt]`): Affine coordinates (x, y) representing $Acc_Point_{\mathcal{E}_2}$ — stored as raw integers because Ergo's `GroupElement` type validates exclusively against the native curve
- `R7` (`Int`): $\text{ChunkIndex } k \in [0, N-1]$

4.3 Permissionless Cranking & Deterministic Off-Chain Simulation

A naive pipeline design requires the user to pre-sign and broadcast all $N = 1,200$ transactions simultaneously. However, Ergo UTXO identifiers are rigorously deterministic ($\text{Box.id} = \text{Hash}(Tx.id, output_index)$). If a fee-market spike occurs and Tx_5 stalls in the mempool, utilizing "Replace-by-Fee" mutates $Tx_5.id$, instantly invalidating the pre-signed Tx_6 through Tx_{1200}.

To ensure topological resilience against mempool lockouts, the pipeline must utilize **Permissionless Cranking**. The `ProofInProgress` contract asserts no conditions regarding the identity of the transaction submitter—it only enforces that the mathematical state continuation is valid. The prover attaches an "Execution Bounty" (denominated strictly in ERG to eliminate foreign-exchange volatility risk for relayers) to the final state, sized to amortize the cumulative miner fees across all N transactions (estimated at ≈ 1.2 ERG assuming 0.001 ERG per transaction). If the pipeline stalls, MEV bots and relayers are incentivized to dynamically construct Tx_k, pay the mempool fee out of pocket, and advance the state to claim the bounty.

Execution griefing is mathematically impossible on eUTXO. A critical property exploited by this design is that Ergo's eUTXO model is strictly deterministic: the validity and outcome of a transaction are fully determined by its inputs, outputs, and context at construction time [6]. Unlike account-based models where global state mutations can unpredictably alter execution paths between transaction creation and mining, an eUTXO script evaluates identically off-chain and on-chain. Consequently, an MEV operator intercepting Tx_1 in the mempool can extract the proof transcript from its registers, load the cached SRS generators (which are static public parameters, stored locally after a one-time download from the Genesis AVL Tree), and execute the entire Generalized Bulletproof verification locally on commodity hardware in sub-second latency. If the proof is mathematically invalid, the bot drops the transaction before committing any capital. The probability of a rational MEV bot initiating a pipeline destined to fail is therefore zero, entirely eliminating the perceived need for slashable collateral bonds—an Ethereum-inherited assumption that does not apply to deterministic eUTXO architectures.

Upon reaching Tx_N, the pipeline mints a `Verified_Proof` token. A separate, single-block transaction can then aggregate dozens of `Verified_Proof` tokens from multiple concurrent users to batch-update the global Curve Tree Accumulator simultaneously, preventing global state lockouts.

Ergo miners, processing this topological directed acyclic graph (DAG) of the eUTXO model, will linearly sequence the transactions into the blockchain across $\approx$ 1,200 chronological blocks. The network acts fundamen-

tally as a decentralized, stateful Turing tape executing exactly $\sim 900,000$ JitCost of alternating curve MSM per block.

4.4 Architectural Ramifications

While the total state transition (Tx_{1200}) will necessitate approximately 24 to 40 hours to achieve absolute cryptographic finality, Ergo's pending Sub-blocks protocol upgrade provides a transformative User Experience (UX) vector.

As Sub-blocks generate Weak Confirmations on scales of ≈ 1 second, the client UI provides deterministic, granular feedback as each Tx_k enters a sub-block. This proves instantly that the heavy zero-knowledge accumulator state transition is securely in deterministic motion, mirroring Web2 latencies natively on L1. We emphasize that Sub-blocks reduce *perceived* latency (user-facing confirmation feedback) but do not alter the cryptographic finality time of the full pipeline. Until the Sub-blocks upgrade is deployed, the ≈ 40-hour finality window is an irreducible operational constraint.

5. Related Work

Contemporary blockchains approach recursive proof compositions through varied architectures. Zcash employs the Halo 2 protocol over the Pasta cycle (Pallas and Vesta) natively at the protocol layer, requiring specific cryptographic parameters embedded into consensus. Mina utilizes a recursive SNARK structure (Pickles) operating on specialized curves to maintain a constant-size blockchain. In contrast, our proposed architecture requires *zero protocol-level modifications*, achieving similar highly scalable zero-knowledge accumulators purely via smart contract algorithmic simulation and the native eUTXO topology.

6. Conclusion and Future Work

By uniting the efficient Generalized Bulletproofs arithmetic scheme, the 2-chain completion via the `secq256k1` `UnsignedBigInt` simulation, and the topological properties of eUTXO Execution Pipelining, Ergo L1 is mathematically equipped to support massive-scale Zero-Knowledge Accumulators today.

However, this architecture presents distinct operational limitations. The reliance on sequential multi-block execution introduces latency constraints (≈ 40 hours for full finality). Future work must analyze fee market dynamics under extreme network congestion, nullifier state-machine topologies for specific privacy pool implementations, and the interaction of Execution Pipelining with Ergo's pending Sub-blocks upgrade. Additionally, empirical benchmarking of `UnsignedBigInt` opcode costs on testnet infrastructure would refine the pipeline length estimates presented in §3.4. Nonetheless, this theoretical reality is achievable cleanly at the current smart-contract layer without any prerequisite soft forks or core protocol mutations.

References

[1] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrien Rodriguez. "Curve Trees: Practical and Transparent Zero-Knowledge Accumulators." *IACR Cryptol. ePrint Arch.*, 2022. Available: https://eprint.iacr.org/2022/756

[2] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. "Bulletproofs: Short Proofs for Confidential Transactions and More." *IEEE Symposium on Security and Privacy (S&P)*, 2018. Available: https://eprint.iacr.org/2017/1066

[3] Alexander Chepurnoy and Dmitry Meshkov. "Ergo: A Resilient Platform for Financial Contracts." *Ergo Platform Whitepaper*, 2019. Available: https://ergoplatform.org/en/documents/

[4] Sean Bowe, Jack Grigg, and Daira Hopwood. "Recursive Proof Composition without a Trusted Setup." *IACR Cryptol. ePrint Arch.*, 2019. Available: https://eprint.iacr.org/2019/1021

[5] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. "Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting." *Eurocrypt*, 2016. Available: https://eprint.iacr.org/2016/263

[6] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Michael Peyton Jones, and Philip Wadler. "The Extended UTXO Model." *International Conference on Financial Cryptography and Data Security*, 2020. Available: https://eprint.iacr.org/2020/231

[7] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves." *Algorithmica*, 2014. Available: https://eprint.iacr.org/2014/595

Notes

[1] `CryptoConstants.groupOrder` is exposed in ErgoScript 6.0 as a built-in constant of type `UnsignedBigInt`, accessible without explicit import. If future node versions restrict its availability, the 256-bit prime n_{secp} = `0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141` can be hardcoded as a literal.

[2] These costs are derived from the EIP-0050 specification. Empirical validation on Ergo testnet v6.0.1 is underway; if measured costs diverge from the specification, the pipeline length estimates in §3.4 and §4.2 would shift proportionally.